NLP-nameOrigins : Documentation Technique Complète

Étudiant NLP – Version 1.0, Mars 2026

Table des matières

1. Présentation générale du projet
   1.1. Contexte et objectifs
   1.2. Problématique
   1.3. Architecture globale du pipeline
2. Environnement de développement
   2.1. Configuration technique
   2.2. Configuration de l'API Mistral
   2.3. Dépendances
   2.4. Structure des fichiers
3. Sources de données
   3.1. Geneanet — names.json et origins.json
      3.1.1. Présentation
      3.1.2. Structure de names.json
      3.1.3. Structure de origins.json
   3.2. INSEE — noms2008nat_txt.txt
      3.2.1. Présentation
      3.2.2. Structure
      3.2.3. Points d'attention
   3.3. prenoms.com — Scraping web
4. Étape 1 — Clustering par Union-Find
   4.1. Objectif
   4.2. Algorithme Union-Find
   4.3. Application au projet
   4.4. Résultats
   4.5. Format de sortie — clusters.json
5. Étape 2 — Résumés par Mistral AI
   5.1. Objectif
   5.2. Stratégie
   5.3. Prompt utilisé
   5.4. Format de sortie — final_database.json
6. Étape 3 — Scraping des prénoms
   6.1. Objectif
   6.2. Méthode de scraping
      6.2.1. Découverte des URLs via sitemaps
      6.2.2. Extraction du contenu
   6.3. Résultats
   6.4. Format de sortie — prenoms_database.json
7. Étape 4 — Intégration des données INSEE
   7.1. Objectif
   7.2. Chargement du fichier INSEE
   7.3. Croisement avec Geneanet
   7.4. Résultats du croisement
   7.5. Top 10 des noms les plus fréquents
8. Étape 5 — Correspondances phonétiques et orthographiques
   8.1. Objectif
   8.2. Algorithme Soundex
      8.2.1. Construction de l'index Soundex Geneanet
   8.3. Distance de Levenshtein
   8.4. Pipeline de traitement
   8.5. Résultats
   8.6. Format de sortie — full_database.json
9. Étape 6 — Extraction NER
   9.1. Objectif
   9.2. Méthode 1 — Regex + majuscules
      9.2.1. Principe
      9.2.2. Liste noire (MOTS_A_IGNORER)

9.3. Méthode 2 — Mistral AI (référence)
9.3.1. Correction critique — parsing JSON
9.4. Méthode combinée — Regex + validation Mistral
9.5. Évaluation sur 200 textes
10. Étape 7 — Normalisation et variantes régionales
10.1. Partie 1 — Noms composés
10.1.1. Problème
10.1.2. Solution — Normalisation par agglutination
10.1.3. Résultats
10.2. Partie 2 — Variantes régionales
10.2.1. Méthode
10.2.2. Résultats
11. Interface de recherche
11.1. Interface ligne de commande — recherche.py
11.2. Interface web — app.py (Streamlit)
11.2.1. Architecture
11.2.2. Gestion de l'état (session_state)
11.2.3. Fonctionnalités de l'interface
12. Bilan et évaluation globale
12.1. Récapitulatif par étape
12.2. Couverture finale de la base
12.3. Analyse des performances NER
13. Perspectives d'amélioration
14. Annexes
14.1. Annexe A — Commandes d'utilisation
14.2. Annexe B — Exemples de recherches
14.3. Annexe C — Format complet full_database.json

Documentation technique complète du projet **NLP-nameOrigins** — construction automatique d'une base étymologique des noms de famille et prénoms français par des techniques de traitement automatique du langage naturel.

---

1. Présentation générale du projet

1.1. Contexte et objectifs

Le projet **NLP-nameOrigins** s'inscrit dans le cadre d'un travail pratique de NLP. L'objectif central est d'exploiter une base de données de noms de famille français avec leurs textes étymologiques pour :

- Identifier automatiquement les variantes orthographiques et phonétiques d'un même patronyme
- Regrouper ces variantes en clusters cohérents
- Enrichir la base avec des données de fréquence INSEE
- Étendre la base aux prénoms via scraping web
- Offrir une interface de recherche conviviale

1.2. Problématique

> *Comment identifier automatiquement les variantes d'un nom de famille français et enrichir une base étymologique incomplète à partir de sources hétérogènes ?*

La difficulté centrale est qu'un même patronyme peut s'écrire de multiples façons selon les régions, les époques et les migrations :

Type de variation	Exemples	Cause
Phonétique	Durrand → Durand	Erreur orthographique historique
Régionale	Lefebvre / Lefeuvre / Lefèvre	Dialectes régionaux
Composés	Le Goff / Legoff / Le-Goff	Agglutination ou séparation
Latinisation	Durandus → Durand	Registres ecclésiastiques

Migration	Garcia / Garcias	Adaptation phonétique

1.3. Architecture globale du pipeline

Le projet est organisé en 7 étapes séquentielles :

```text
names.json + origins.json (Geneanet)
          │
          ▼
[Étape 1] Union-Find ──────────────────► clusters.json
          │
          ▼
[Étape 2] Résumés Mistral AI ──────────► final_database.json
          │
          ▼                    prenoms.com (scraping)
[Étape 3] Scraping prénoms ────────────► prenoms_database.json
          │
          ▼            noms2008nat_txt.txt (INSEE)
[Étape 4] Intégration INSEE ───────────► insee_database.json
          │
          ▼
[Étape 5] Soundex + Levenshtein ───────► full_database.json (218 982 entrées)
          │
          ▼
[Étape 6] NER extraction ──────────────► nouveaux_noms.json + evaluation.json
          │
          ▼
[Étape 7] Normalisation ───────────────► full_database_v2.json + variantes_regionales.json
          │
          ▼
[app.py]  Interface Streamlit ◄──────────────────────────────────────────
```

---

2. Environnement de développement

2.1. Configuration technique

- **Système d'exploitation** : Windows 10/11
- **IDE** : Visual Studio Code
- **Langage** : Python 3.12
- **Environnement virtuel** : `venv` (Windows PowerShell)
- **Modèle LLM** : `mistral-large-latest` via API Mistral
- **Interface** : Streamlit 1.x

2.2. Configuration de l'API Mistral

Le projet utilise le client OpenAI avec l'URL de base de Mistral :

```python
from openai import OpenAI
from dotenv import load_dotenv
import os

load_dotenv()

client = OpenAI(
    api_key=os.getenv("MISTRAL_API_KEY"),
    base_url="https://api.mistral.ai/v1"
)

response = client.chat.completions.create(
    model="mistral-large-latest",
    messages=[{"role": "user", "content": prompt}],
    max_tokens=500
)
```

Fichier `.env` requis :

```text
MISTRAL_API_KEY=sk-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

2.3. Dépendances

```text
openai>=1.0.0         # Client API Mistral (compatible OpenAI)
python-dotenv         # Chargement des variables d'environnement
requests              # Requêtes HTTP pour le scraping
beautifulsoup4        # Parsing HTML
jellyfish             # Soundex et algorithmes phonétiques
python-Levenshtein    # Distance de Levenshtein optimisée (C)
rapidfuzz             # Correspondances floues rapides
streamlit             # Interface web
plotly                # Graphiques interactifs
pandas                # Manipulation de données
```

2.4. Structure des fichiers

```text
NLP-nameOrigins/
├── data/
│   ├── names.json                  ← 21 777 noms de famille (fourni)
│   ├── origins.json                ← 20 981 textes étymologiques (fourni)
│   └── noms2008nat_txt.txt         ← 218 982 noms INSEE
├── output/
│   ├── clusters.json               ← groupes Union-Find
│   ├── final_database.json         ← base Geneanet résumée
│   ├── prenoms_raw.json            ← prénoms bruts (intermédiaire)
│   ├── prenoms_database.json       ← prénoms avec origines
│   ├── insee_database.json         ← base INSEE enrichie
│   ├── full_database.json          ← base complète 218 982 entrées
│   ├── full_database_v2.json       ← base avec noms composés fusionnés
│   ├── nouveaux_noms.json          ← variantes extraites par NER
│   ├── evaluation.json             ← métriques d'évaluation NER
│   └── variantes_regionales.json   ← variantes régionales confirmées
├── etape1_clustering.py
├── etape2_summarize.py
├── etape3_prenoms.py
├── etape4_insee.py
├── etape5_phonetique.py
├── etape6_extraction_noms.py
├── etape7_normalisation.py
├── recherche.py
├── app.py
├── test_mistral.py
├── .env
├── .gitignore
└── requirements.txt
```

---

3. Sources de données

3.1. Geneanet — names.json et origins.json

3.1.1. Présentation

Les deux fichiers de base sont issus de **Geneanet**, site de généalogie collaboratif français, et plus précisément du dictionnaire étymologique de **Jean Tosti**, onomasticien spécialisé.

3.1.2. Structure de names.json

```json
[
  {"name": "aabdi",   "origins": ["012516"]},
  {"name": "aalberg", "origins": ["05286"]},
  {"name": "aaron",   "origins": ["05171"]},
  ...
]
```

- **Type** : liste de 21 777 objets
- **Champ** `name` : nom de famille en minuscules
- **Champ** `origins` : liste d'identifiants d'origine (en général un seul, parfois plusieurs)

3.1.3. Structure de origins.json

```json
{
  "01": "Variante de Bidault (voir ce nom) assez fréquente en Bretagne...",
  "02": "Nom de personne d'origine germanique...",
  ...
}
```

- **Type** : dictionnaire de 20 981 entrées
- **Clé** : identifiant `0xxxx`
- **Valeur** : texte étymologique en français (2 à 10 phrases en moyenne)

> Les textes mentionnent fréquemment des variantes orthographiques, des régions géographiques, des formes historiques et des chiffres de fréquence INSEE.

3.2. INSEE — noms2008nat_txt.txt

3.2.1. Présentation

Fichier officiel de l'INSEE contenant les fréquences des noms de famille par décennie depuis 1891.

- **URL** : https://www.insee.fr/fr/statistiques/3536630
- **Encodage** : `latin-1`
- **Délimiteur** : tabulation (`\t`)
- **Taille** : 218 983 lignes (dont 1 en-tête)

3.2.2. Structure

```
NOM          _1891_1900  _1901_1910  ...  _1991_2000
MARTIN       12453       13201       ...  18742
BERNARD      7823        8102        ...  9341
...
```

Colonnes disponibles :

Colonne	Description
`NOM`	Nom de famille en majuscules
`_1891_1900`	Naissances portant ce nom entre 1891 et 1900
`_1901_1910`	Naissances entre 1901 et 1910
(8 décennies supplémentaires)	…
`_1991_2000`	Naissances entre 1991 et 2000

3.2.3. Points d'attention

- La ligne `"Autres noms"` totalise les noms rares — elle est **filtrée explicitement**
- La fréquence totale = somme de toutes les décennies
- Certains noms n'ont des données que sur quelques décennies

3.3. prenoms.com — Scraping web

- **Site** : https://www.prenoms.com
- **Structure** : sitemaps XML par lettre (`/sitemap-prenoms-A.xml` … `Z.xml`)
- **Contenu** : page individuelle par prénom avec texte d'origine, genre, variantes

---

4. Étape 1 — Clustering par Union-Find

4.1. Objectif

Regrouper automatiquement les noms de famille qui partagent le même texte d'origine. Si `durand` et `durant` partagent l'identifiant `01234` , ils sont variantes l'un de l'autre.

4.2. Algorithme Union-Find

L'algorithme Union-Find (ou **Disjoint Set Union**) est une structure de données qui maintient une partition d'un ensemble en groupes disjoints. Il supporte deux opérations en quasi-temps constant :

- **Find(x)** : trouver le représentant du groupe de x
- **Union(x, y)** : fusionner les groupes de x et y

```python
class UnionFind:
    def __init__(self, elements):
        self.parent = {e: e for e in elements}
        self.rank   = {e: 0 for e in elements}

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])  # compression de chemin
        return self.parent[x]

    def union(self, x, y):
        rx, ry = self.find(x), self.find(y)
        if rx == ry:
            return
        if self.rank[rx] < self.rank[ry]:
            rx, ry = ry, rx
        self.parent[ry] = rx
        if self.rank[rx] == self.rank[ry]:
            self.rank[rx] += 1
```

4.3. Application au projet

```python
# Construire l'index : origin_id → liste de noms
origin_to_names = defaultdict(list)
for entry in names:
    for oid in entry["origins"]:
        origin_to_names[oid].append(entry["name"])

# Appliquer Union-Find
uf = UnionFind(all_names)
for oid, noms in origin_to_names.items():
    for i in range(1, len(noms)):
        uf.union(noms[0], noms[i])

# Construire les clusters
clusters = defaultdict(list)
for nom in all_names:
    clusters[uf.find(nom)].append(nom)
```

4.4. Résultats

Métrique	Valeur
Total de clusters	20 597
Clusters isolés (1 seul nom)	19 567
Groupes de variantes (2+ noms)	**1 030**
Taille maximale d'un cluster	6
Exemple de cluster	duran / durand / durant / duranton / duranteau

4.5. Format de sortie — clusters.json

```json
[
  {
    "representative": "durand",
    "members": ["duran", "durand", "durant", "duranton"],
    "origin_ids": ["01234"]
  },
  ...
]
```

---

5. Étape 2 — Résumés par Mistral AI

5.1. Objectif

Pour les 372 clusters ayant plusieurs textes d'origine distincts, produire un texte de synthèse unique sans perte d'information.

5.2. Stratégie

Cas	Traitement
Cluster avec 1 seul texte (20 225 cas)	Copie directe du texte — pas d'appel API

Cluster avec N textes distincts (372 cas)	Appel Mistral AI pour résumé fusionné

5.3. Prompt utilisé

```python
prompt = f"""Tu es un expert en onomastique française.
Voici {n} textes d'origine pour des variantes du même nom de famille.
Rédige un seul paragraphe de synthèse qui :
- Conserve toutes les informations importantes
- Mentionne toutes les variantes orthographiques
- Cite les régions géographiques mentionnées
- Reste factuel et précis

Textes :
{chr(10).join(textes)}

Réponds uniquement avec le texte synthétisé, sans introduction."""
```

5.4. Format de sortie — final_database.json

```json
[
  {
    "representative": "durand",
    "members": ["duran", "durand", "durant", "duranton"],
    "origin_text": "Nom de personne d'origine germanique, dérivé de *dur*..."
  },
  ...
]
```

---

6. Étape 3 — Scraping des prénoms

6.1. Objectif

Construire une base de prénoms français analogue à la base de noms de famille, en scrapant prenoms.com.

6.2. Méthode de scraping

6.2.1. Découverte des URLs via sitemaps

```python
import requests
from bs4 import BeautifulSoup

SITEMAPS = [
    f"https://www.prenoms.com/sitemap-prenoms-{lettre}.xml"
    for lettre in "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
]

def get_urls_from_sitemap(sitemap_url):
    r = BeautifulSoup(requests.get(sitemap_url).text, "xml")
    return [loc.text for loc in r.find_all("loc")]
```

6.2.2. Extraction du contenu

```python
def scraper_prenom(url):
    soup = BeautifulSoup(requests.get(url).text, "html.parser")

    # Nom du prénom
    prenom = soup.find("h1").text.strip().lower()

    # Genre
    genre_tag = soup.find("span", class_="gender")
    genre = genre_tag.text.strip() if genre_tag else ""

    # Texte d'origine
    origine_tag = soup.find("div", class_="origin-text")
    texte = origine_tag.get_text(separator=" ").strip() if origine_tag else ""

    # Variantes mentionnées dans le texte
    variantes = extraire_variantes(texte)

    return {"prenom": prenom, "genre": genre, "origin_text": texte,
            "variants": variantes}
```

6.3. Résultats

Métrique	Valeur
Prénoms uniques scrapés	11 455

Groupes de variantes détectés	19
Résumés Mistral générés	19
Exemple de groupe	jean / johanne / jeannot

6.4. Format de sortie — prenoms_database.json

```JSON
[
  {
    "variants": ["jean", "johanne", "jeannot"],
    "genre": "garcon",
    "origin_text": "Prénom d'origine hébraïque, forme française de Johannes..."
  },
  ...
]
```

---

7. Étape 4 — Intégration des données INSEE

7.1. Objectif

Croiser les 218 982 noms de la base INSEE avec la base Geneanet pour associer fréquences et origines.

7.2. Chargement du fichier INSEE

```PYTHON
import csv

noms_insee = {}
with open("data/noms2008nat_txt.txt", encoding="latin-1") as f:
    reader = csv.DictReader(f, delimiter="\t")
    for row in reader:
        nom = row["NOM"].strip().lower()

        # Filtrer la ligne "Autres noms"
        if nom == "autres noms":
            continue

        frequence = sum(
            int(v) for k, v in row.items()
            if k != "NOM" and v.isdigit()
        )
        evolution = {
            k: int(v) for k, v in row.items()
            if k != "NOM" and v.isdigit()
        }
        noms_insee[nom] = {
            "frequence": frequence,
            "evolution": evolution
        }
```

7.3. Croisement avec Geneanet

```PYTHON
# Index Geneanet : nom → entrée
index_geneanet = {}
for cluster in final_database:
    for membre in cluster["members"]:
        index_geneanet[membre] = cluster

# Croisement
resultats = {"avec_origine": 0, "sans_origine": 0}
for nom, data in noms_insee.items():
    if nom in index_geneanet:
        resultats["avec_origine"] += 1
    else:
        resultats["sans_origine"] += 1
```

7.4. Résultats du croisement

Catégorie	Nombre	Pourcentage
Noms avec origine Geneanet directe	17 211	7.9%
Noms sans origine (à traiter étapes 5+)	201 771	92.1%
Total noms INSEE	**218 982**	**100%**

> Seulement 7.9% de correspondance directe car Geneanet ne couvre que les noms les plus courants avec des

> textes détaillés. Les 92.1% restants seront traités par les algorithmes phonétiques et orthographiques.

7.5. Top 10 des noms les plus fréquents

Rang	Nom	Fréquence totale
1	Martin	250 013
2	Bernard	131 330
3	Thomas	118 331
4	Petit	109 742
5	Robert	106 851
6	Richard	104 892
7	Durand	103 421
8	Dubois	99 843
9	Moreau	97 210
10	Laurent	94 103

---

8. Étape 5 — Correspondances phonétiques et orthographiques

8.1. Objectif

Pour les 201 771 noms INSEE sans correspondance Geneanet directe, trouver un nom Geneanet proche par la phonétique (Soundex) ou l'orthographe (Levenshtein).

8.2. Algorithme Soundex

L'algorithme **Soundex** encode un mot en un code phonétique de 4 caractères : la première lettre suivie de 3 chiffres représentant les consonnes.

```python
import jellyfish

jellyfish.soundex("DURAND")   # → "D653"
jellyfish.soundex("DURANT")   # → "D653"  (même code !)
jellyfish.soundex("DURAN")    # → "D650"
jellyfish.soundex("MARTIN")   # → "M635"
jellyfish.soundex("MARTINEZ") # → "M635"  (même code !)
```

Règles de codage Soundex :

Chiffre	Consonnes représentées
1	B, F, P, V
2	C, G, J, K, Q, S, X, Z
3	D, T
4	L
5	M, N
6	R

8.2.1. Construction de l'index Soundex Geneanet

```python
# Index : code_soundex → liste de représentants Geneanet
soundex_index = defaultdict(list)
for cluster in final_database:
    rep = cluster["representative"]
    code = jellyfish.soundex(rep.upper())
    soundex_index[code].append(rep)
# Résultat : 2 913 codes Soundex distincts
```

8.3. Distance de Levenshtein

La **distance de Levenshtein** mesure le nombre minimal d'opérations (insertion, suppression, substitution) pour transformer une chaîne en une autre.

```python
from Levenshtein import distance

distance("durand",  "durrand")  # → 1  (suppression d'un 'r')
distance("lefebre", "lefebvre") # → 1  (insertion d'un 'v')
distance("martin",  "martins")  # → 1  (suppression du 's')
distance("dupont",  "dupond")   # → 1  (substitution t→d)
```

8.4. Pipeline de traitement

```python
for nom_insee in noms_sans_origine:
    # 1. Essai Soundex
    code = jellyfish.soundex(nom_insee.upper())
    candidats = soundex_index.get(code, [])
    if candidats:
        meilleur = min(candidats, key=lambda c: distance(nom_insee, c))
        if distance(nom_insee, meilleur) <= 3:
            assigner_origine(nom_insee, meilleur, source="soundex")
            continue

    # 2. Essai Levenshtein étendu (distance ≤ 2)
    for rep_geneanet in tous_les_representants:
        if distance(nom_insee, rep_geneanet) <= 2:
            assigner_origine(nom_insee, rep_geneanet, source="levenshtein")
            break

    # 3. Génération par Mistral (si toujours sans origine)
    else:
        origine = generer_origine_mistral(nom_insee)
        assigner_origine(nom_insee, nom_insee, source="mistral_generated",
                         texte=origine)
```

8.5. Résultats

Méthode	Noms couverts	Pourcentage
Geneanet direct (étape 4)	17 211	7.9%
Soundex	183 471	83.8%
Levenshtein	10 001	4.6%
Mistral AI généré	1 000	0.5%
Non couverts	**7 299**	**3.3%**
TOTAL COUVERT	**211 683**	**96.7%**

8.6. Format de sortie — full_database.json

```JSON
{
  "nom": "durrand",
  "variantes": ["duran", "durand", "durant", "durrand"],
  "frequence": 108374,
  "evolution": {
    "_1891_1900": 4521,
    "_1901_1910": 5832,
    "_1911_1920": 6103,
    "_1921_1930": 7841,
    "_1931_1940": 8920,
    "_1941_1950": 10234,
    "_1951_1960": 14521,
    "_1961_1970": 18432,
    "_1971_1980": 16230,
    "_1981_1990": 12421,
    "_1991_2000": 3319
  },
  "origin_text": "Nom de personne d'origine germanique...",
  "source": "soundex",
  "correspond_a": "durand"
}
```

---

9. Étape 6 — Extraction NER

9.1. Objectif

Les textes étymologiques mentionnent souvent des variantes et des noms associés qui ne figurent pas encore dans la base. L'objectif est de les extraire automatiquement.

Exemple de texte source :

```TEXT
"Variante de Bidault (voir ce nom) assez fréquente en Bretagne (29, 56).
Formes voisines : Bideaud (25, 38, 44), Bideault, Bideaut (71), Bideaux (02, 76, 90)."
```

Noms à extraire : `Bidault`, `Bideaud`, `Bideault`, `Bideaut`, `Bideaux`

9.2. Méthode 1 — Regex + majuscules

9.2.1. Principe

Détecter les tokens en majuscules précédés de mots-clés indicateurs de variantes :

```PYTHON
MOTS_CLES = [
    "Variante de", "Voir aussi", "Voir ce nom",
    "Formes voisines", "Forme voisine",
    "Également écrit", "On trouve aussi",
    "Autres formes", "Forme bretonne",
]

PATTERN_NOM = r'\b[A-ZÀ-Ü][a-zà-ü]{2,}(?:\s[A-ZÀ-Ü][a-zà-ü]+)*\b'

def extraire_regex(texte):
    candidats = []
    for mot_cle in MOTS_CLES:
        idx = texte.find(mot_cle)
        if idx != -1:
            extrait = texte[idx:idx+200]
            candidats += re.findall(PATTERN_NOM, extrait)
    return list(set(candidats))
```

9.2.2. Liste noire (MOTS_A_IGNORER)

Les faux positifs sont filtrés via une liste noire de ~300 mots :

```PYTHON
MOTS_A_IGNORER = {
    # Régions et pays
    "Bretagne", "Normandie", "Alsace", "France", "Espagne",
    # Siècles et dates
    "Moyen", "Siècle", "Ancien", "Moderne",
    # Mots grammaticaux
    "Voir", "Forme", "Variante", "Nom", "Prénom",
    # Auteurs et sources
    "Tosti", "Geneanet", "INSEE",
    # ... (~300 entrées au total)
}
```

9.3. Méthode 2 — Mistral AI (référence)

Mistral reçoit le texte brut et doit retourner une liste JSON de noms propres :

```python
prompt = f"""Extrais tous les noms de famille mentionnés dans ce texte.
Retourne UNIQUEMENT un tableau JSON de chaînes, sans aucun autre texte.

Texte : {texte}"""

response = client.chat.completions.create(
    model="mistral-large-latest",
    messages=[{"role": "user", "content": prompt}],
    max_tokens=200
)
```

9.3.1. Correction critique — parsing JSON

Mistral retourne parfois des blocs Markdown :

```python
# ❌ Problème : réponse avec backticks
# ```json
# ["Bidault", "Bideaud", "Bideault"]
# ```

# ✅ Solution : supprimer les balises avant json.loads()
texte_reponse = response.choices[0].message.content
texte_reponse = texte_reponse.replace("```json", "").replace("```", "").strip()
noms = json.loads(texte_reponse)
```

9.4. Méthode combinée — Regex + validation Mistral

Les candidats extraits par Regex sont soumis à Mistral pour validation :

```python
prompt = f"""Parmi ces mots extraits d'un texte étymologique,
lesquels sont de vrais noms de famille français ?
Réponds UNIQUEMENT avec un tableau JSON des noms valides.

Candidats : {json.dumps(candidats_regex)}"""
```

9.5. Évaluation sur 200 textes

Méthode	Précision	Rappel	F1-score
Regex + majuscules seul	51.0%	60.0%	55.1%
Combinée (Regex + Mistral)	**86.9%**	**57.5%**	**69.2%**
Mistral seul *(référence)*	100%	100%	100%

Analyse des résultats :

- La méthode combinée multiplie la précision par 1.7 (+35.9 pts) par rapport au Regex seul
- Le rappel reste limité à 57.5% car certaines variantes n'ont pas de marqueur textuel explicite
- Mistral seul sert de référence théorique (trop coûteux à appliquer aux 20 981 textes)

---

10. Étape 7 — Normalisation et variantes régionales

10.1. Partie 1 — Noms composés

10.1.1. Problème

Les noms composés existent sous de multiples formes graphiques qui ne sont pas reconnues comme identiques :

- `Le Goff`, `Legoff`, `Le-Goff`
- `Du Bois`, `Dubois`, `Du-Bois`
- `Le Roy`, `Leroy`, `Le-Roy`

10.1.2. Solution — Normalisation par agglutination

```python
def normaliser_compose(nom):
    return nom.lower() \
              .replace("-", "") \
              .replace(" ", "") \
              .replace("'", "")

# Exemples :
# "le goff"  → "legoff"
# "Le-Goff"  → "legoff"
# "LEGOFF"   → "legoff"
# → toutes identiques après normalisation
```

10.1.3. Résultats

- **2 688 groupes** de noms composés fusionnés
- Exemples : `dubois/du bois`, `leroy/le roy/le-roy`, `legoff/le goff/le-goff`

10.2. Partie 2 — Variantes régionales

10.2.1. Méthode

Les groupes Soundex candidats sont soumis à Mistral pour valider s'ils constituent de vraies variantes régionales :

```python
prompt = f"""Ces noms de famille ont le même code Soundex.
Sont-ils de vraies variantes régionales d'un même patronyme ?
Réponds JSON : {{"est_variante": true/false, "explication": "..."}}

Noms : {json.dumps(groupe)}"""
```

10.2.2. Résultats

Métrique	Valeur
Groupes Soundex candidats analysés	100 (sur 3 528 total)
Confirmés comme variantes régionales	31
Taux de confirmation	31%

Exemples de variantes régionales confirmées :

Groupe	Région / Explication
lefebvre / lefevre / lefeuvre	Nord-Pas-de-Calais / Normandie / Bretagne
durand / durant / duranton	Toute la France / Sud-Ouest / Centre
petit / petiot / petiteau	Bourgogne / Centre
martin / martins / martinez	France / Portugal / Espagne

---

11. Interface de recherche

11.1. Interface ligne de commande — recherche.py

```python
def rechercher(query, index_noms, index_prenoms):
    mots = query.strip().split()

    if len(mots) == 1:
        # Recherche nom ou prénom
        res_nom    = trouver_dans_index(mots[0], index_noms)
        res_prenom = trouver_dans_index(mots[0], index_prenoms)
        return res_nom, res_prenom

    elif len(mots) == 2:
        # Essai prénom + nom
        res_p = trouver_dans_index(mots[0], index_prenoms)
        res_n = trouver_dans_index(mots[1], index_noms)
        if res_p and res_n:
            return res_n, res_p
        # Essai nom + prénom
        ...
```

Normalisation pour la recherche :

```python
def normaliser_recherche(texte):
    texte = texte.strip().lower()
    # Supprimer les accents
    texte = unicodedata.normalize('NFD', texte)
    texte = ''.join(c for c in texte if unicodedata.category(c) != 'Mn')
    return texte

# "Dubois" → "dubois"
# "Hébert"  → "hebert"
# "Le Goff" → "le goff" → puis index normalisé
```

11.2. Interface web — app.py (Streamlit)

11.2.1. Architecture

```text
app.py
├── charger_donnees()          ← cache @st.cache_data
├── normaliser(texte)
├── rechercher(query, index)
├── trouver_proches(nom, n)    ← Soundex
├── graphique_evolution(entree) ← Plotly bar chart
├── graphique_carte(entree)    ← concentration régionale
├── saviez_vous(entree)        ← anecdote automatique
├── afficher_nom(entree)       ← card résultat nom
├── afficher_prenom(entree)    ← card résultat prénom
└── main()
    ├── Sidebar : stats + mode sombre
    ├── Tab 1 : Recherche
    │   ├── Barre de recherche
    │   ├── Boutons exemples
    │   ├── Historique (session_state)
    │   ├── Saviez-vous ?
    │   ├── Résultats (nom + prénom)
    │   ├── Noms proches
    │   └── Graphiques
    └── Tab 2 : Comparer
        ├── Deux champs de saisie
        ├── Affichage côte à côte
        └── Métriques comparatives
```

11.2.2. Gestion de l'état (session_state)

```python
# Historique des recherches persistant dans la session
def ajouter_historique(query):
    if "historique" not in st.session_state:
        st.session_state.historique = []
    if query not in st.session_state.historique:
        st.session_state.historique.insert(0, query)
        st.session_state.historique = st.session_state.historique[:5]
```

11.2.3. Fonctionnalités de l'interface

Fonctionnalité	Description
Recherche unifiée	Accepte nom seul, prénom seul, ou "prénom nom" / "nom prénom"
Suggestions phonétiques	Noms phonétiquement proches via Soundex (boutons cliquables)
Historique de session	5 dernières recherches, cliquables
Anecdote automatique	"Saviez-vous ?" généré selon les données du nom
Graphique évolution	Bar chart Plotly des naissances par décennie (INSEE 1891-2000)
Mode sombre	Toggle dans la sidebar, CSS dynamique
Comparateur	Distance Levenshtein + Soundex + fréquence entre deux noms
Statistiques sidebar	Métriques complètes de la base (pour présentation enseignant)

---

12. Bilan et évaluation globale

12.1. Récapitulatif par étape

Étape	Technique	Entrée	Sortie
1 — Clustering	Union-Find	21 777 noms + 20 981 origines	1 030 groupes de

			1 030 groupes de variantes
2 — Résumés	Mistral AI	372 clusters multi-textes	372 résumés fusionnés
3 — Prénoms	Scraping BeautifulSoup	prenoms.com	11 455 prénoms + origines
4 — INSEE	Lecture fichier TSV	218 982 noms INSEE	Fréquences décennales
5 — Phonétique	Soundex + Levenshtein + Mistral	201 771 noms sans origine	96.7% couverts
6 — NER	Regex + Mistral	20 981 textes	F1 = 69.2%
7 — Normalisation	Agglutination + Soundex + Mistral	full_database.json	2 688 fusions + 31 variantes régionales

12.2. Couverture finale de la base

Métrique	Valeur
Noms de famille total	218 982
Noms avec origine étymologique	211 683 (96.7%)
Prénoms	11 455
Personnes couvertes (fréquences)	73 470 900
Groupes de variantes identifiés	1 030 + 2 688 composés + 31 régionaux

12.3. Analyse des performances NER

Le F1-score de 69.2% de la méthode combinée reflète un compromis entre :

- **Précision élevée (86.9%)** : Mistral filtre efficacement les faux positifs du Regex
- **Rappel limité (57.5%)** : les variantes implicites (sans marqueur textuel) restent non détectées

Pour améliorer le rappel, une piste serait d'utiliser un modèle de NER fine-tuné sur des textes étymologiques français.

---

13. Perspectives d'amélioration

Amélioration	Description	Priorité
Données post-2000	Intégrer les données INSEE jusqu'en 2025	Haute
Carte choroplèthe	Vraie carte de France par département avec heatmap	Haute
NER élargi	Évaluation sur 1 000+ textes pour métriques robustes	Moyenne
Noms étrangers	Espagnols, italiens, arabes, vietnamiens présents en France	Moyenne
Déploiement cloud	Streamlit Cloud ou Hugging Face Spaces	Basse
API REST	Exposer la base via FastAPI	Basse
Fine-tuning NER	Modèle spécialisé sur textes étymologiques	Basse

---

14. Annexes

14.1. Annexe A — Commandes d'utilisation

```bash
# Installation
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt

# Pipeline complet
python etape1_clustering.py
python etape2_summarize.py
python etape3_prenoms.py
python etape4_insee.py
python etape5_phonetique.py
python etape6_extraction_noms.py
python etape7_normalisation.py

# Interfaces
streamlit run app.py      # Interface web → localhost:8501
python recherche.py       # Interface CLI
```

14.2. Annexe B — Exemples de recherches

Requête	Ce qui est trouvé	Source
`martin`	Nom de famille + prénom éventuel	Geneanet direct
`jean`	Prénom Jean + nom de famille Jean (patronyme)	prenoms.com + Geneanet
`jean martin`	Prénom Jean + Nom Martin côte à côte	Combiné
`le goff`	Nom composé breton	Noms composés fusionnés
`durrand`	Trouve "durand" par Levenshtein (distance=1)	Levenshtein
`martinez`	Trouve "martin" par Soundex (M635)	Soundex

14.3. Annexe C — Format complet full_database.json

```json
{
  "nom": "durand",
  "variantes": ["duran", "durand", "durant", "duranton", "duranteau"],
  "frequence": 103421,
  "evolution": {
    "_1891_1900": 3821,
    "_1901_1910": 4932,
    "_1911_1920": 6103,
    "_1921_1930": 7841,
    "_1931_1940": 8920,
    "_1941_1950": 10234,
    "_1951_1960": 14521,
    "_1961_1970": 18432,
    "_1971_1980": 13230,
    "_1981_1990": 11421,
    "_1991_2000": 3966
  },
  "origin_text": "Nom de personne d'origine germanique, dérivé de *dur* (dur, résistant). Très fréquent dans toute la France, surtout dans le Sud-Ouest et le Centre. Formes voisines : Duran (Pyrénées), Durant (Languedoc), Duranton (Berry).",
  "source": "geneanet",
  "correspond_a": "durand"
}
```